CS 300 – Chapter 4 Lectures

Lists, Stacks, Queues, and Deques

Jose de Lima

April 22, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 3 named the seven data structures; chapter 4 picks one family – *linear, ordered collections* – and builds it end-to-end, twice (pointer-based and array-based). Everything you learn here about *cost is a function of memory layout* transfers verbatim to trees (ch. 6), heaps, and graph adjacency lists (opt. ch. 10).

What you add to your toolkit here:

- **Pointer-based lists.** Singly linked (SLL) and doubly linked (DLL); append, prepend, insert-after, remove, search, traverse. You can write these from scratch, by hand.
- **Sentinel / dummy-node idiom.** The trick that collapses "first / middle / last" special cases into one uniform case.
- **Three restricted list ADTs**: Stack (LIFO), Queue (FIFO), Deque (both ends) – each built on linked lists or arrays.
- **Array-based lists.** The `std::vector` cost model (amortized $O(1)$ back, $O(n)$ insert/erase elsewhere) contrasted cleanly with the linked-list cost model.
- **A decision framework** (see 4.18+) for picking between array-based, SLL, DLL, and sentinel-DLL. This is the question that actually shows up on assignments.

Mastery checklist – you should be able to, without references:

1. Draw an SLL of 3 nodes and execute `insertAfter` and `remove` on it by redrawing the pointers one step at a time.
2. Write the node `struct` for an SLL and a DLL; write `append`, `prepend`, `insertAfter`, `remove`, and `search` for both.
3. Explain why `remove` on an SLL requires the *predecessor*'s pointer, and how DLL gets around it.
4. State the cost of every List-ADT operation on (a) SLL, (b) DLL, (c) array-based list – without guessing. Know *why* each one is what it is.
5. Implement Stack, Queue, and Deque on both a linked list and an array, and argue which backing data structure you'd pick for each.
6. Rewrite an SLL method that has a 3-case switch (empty / head / middle) to use a sentinel node and collapse to one case.
7. Traverse a linked list iteratively *and* recursively; know when recursion blows the stack.
8. Given a workload description ("mostly appends, rare middle removals, needs $O(1)$ random access"), pick the right list variant and justify.

Looking ahead. Chapter 5 moves from linear to keyed storage (hash tables), but the pointer-chasing

mental model you're building here is exactly what hash-table chaining uses internally. Chapter 6 (trees) extends the node + pointer idiom from one "next" pointer to two ("left" / "right"); the whole tree chapter will feel like a generalization of this one.

1 4.1 The List ADT

Chapter 3 surveyed the seven fundamental data structures and the nine common abstract data types, then spent most of its energy on searching and sorting. Chapter 4 starts the depth-first tour: each ADT gets its own chapter where we define the contract, implement it, and analyze the operations. First up: the **list**.

The list is the workhorse ADT. Nearly every other sequence-based structure (stack, queue, deque) is a list with additional constraints. Understanding what a list's contract actually *is* – and what it deliberately leaves unspecified – is the first step to picking the right structure later.

Definition — List

A **list** is an abstract data type for a *finite, ordered sequence* of items. Items have *positions* (first, second, last), duplicates are allowed, and operations are defined in terms of those positions: append at the end, prepend at the front, insert after a given item, remove, search, iterate.

Key idea Three things the list ADT guarantees

1. **Order matters.** `(7, 9, 5)` is different from `(5, 7, 9)`. A list is not a set.
2. **Duplicates are allowed.** `(3, 3, 3)` is a valid three-item list.
3. **Positions are stable.** "The second item" refers to a specific slot, not just "any item." Inserting at the front shifts every other item's position index by one; the ADT is aware of this.

1.1 The contract: operations

Operation	What it does	Example (start: 99, 77)
`Append(L, x)`	Insert x at the end	`Append(L, 44)` $\Rightarrow$ 99, 77, 44
`Prepend(L, x)`	Insert x at the front	`Prepend(L, 44)` $\Rightarrow$ 44, 99, 77
`InsertAfter(L, w, x)`	Insert x immediately after w	`InsertAfter(L, 99, 44)` $\Rightarrow$ 99, 44, 77
`Remove(L, x)`	Remove first occurrence of x	`Remove(L, 77)` $\Rightarrow$ 99
`Search(L, x)`	Return item (or `null`)	`Search(L, 99)` $\Rightarrow$ 99; `Search(L, 22)` $\Rightarrow$ null
`Print(L)`	Print items in order	"99, 77"
`PrintReverse(L)`	Print items in reverse order	"77, 99"
`Sort(L)`	Sort items ascending	77, 99
`IsEmpty(L)`	True iff list has no items	`false` for (99, 77)
`GetLength(L)`	Number of items	2

Aside Not every list exposes every operation

The table above is a *menu*, not a mandatory interface. Real list libraries pick a subset. `std::vector` exposes indexed access (`v[i]`) but no constant-time `Prepend`; `std::forward_list` exposes `Prepend` (via `push_front`) but not indexed access. The ADT names the operations; the implementation picks

which to make cheap.

1.2 What the contract *doesn't* say

Key idea Conspicuous silences

The list ADT says nothing about:

- **How items are stored in memory.** Contiguously? Linked nodes? Chunks? All valid.
- **Cost of operations.** The ADT defines what `Append` *means*, not how fast it runs. An array-backed list and a singly-linked list both implement `Append`; they pay very different prices for it.
- **Indexed random access.** The raw list ADT is defined by position-relative operations ("after this item," "first," "last"), not by integer index. The *dynamic array* ADT is the refinement that adds $O(1)$ `at(i)`.

The silences are the point. Leaving implementation unspecified means callers can substitute one list implementation for another without rewriting application code.

Aside Two interface families: Sequence and Set

Step back one level. Every container ADT in CS 300 belongs to one of two families, distinguished by where the order of items lives:

- **Sequence interface.** Items have an *extrinsic* order — the caller chose where each item goes. Operations are positional: `get_at(i)`, `set_at(i,x)`, `insert_at(i,x)`, `insert_first/last(x)`, `delete_first/last()`. *This chapter implements Sequence.* List, stack, queue, and deque are all Sequence with restricted operation sets.
- **Set interface.** Items have an *intrinsic* order determined by a key. Operations are key-based: `find(k)`, `insert(x)`, `delete(k)`, and ordered ops like `find_min()`, `find_next(k)`. ch. 5 (hash tables) and ch. 6 (BSTs) implement Set.

The framing is from MIT 6.006. It explains why the chapter map is the shape it is: ch. 4 builds Sequence implementations end-to-end, then ch. 5–ch. 6 swap the interface and rebuild from scratch. Pick the right family first; pick the implementation second.

1.3 List is the root ADT; stacks and queues restrict it

Definition — Restricted-access lists

Several ADTs later in the chapter are just lists with *restricted* operations:

- **Stack**: a list where you can only insert or remove at *one end* (the "top"). LIFO.
- **Queue**: a list where you insert at one end and remove at the other. FIFO.
- **Deque**: insert or remove at *either* end, but nowhere else.

Narrower interfaces, stronger guarantees about usage patterns, faster implementations possible.

1.4 C++: `std::list`, `std::vector`, `std::forward_list`

C++ gives you several list-like containers. All three satisfy the list ADT; they differ in which operations are cheap and what extra features they offer.

Example Three takes on "list" in C++

```cpp
#include <vector>
#include <list>
#include <forward_list>

std::vector<int>        v = {7, 9, 5};   // dynamic array
std::list<int>          l = {7, 9, 5};   // doubly linked list
std::forward_list<int>  f = {7, 9, 5};   // singly linked list

v.push_back(44);                 // O(1) amortized (append)
l.push_back(44);                 // O(1)
// f has no push_back!           // forward_list is single-link only

v.insert(v.begin(), 44);         // O(n) (prepend: shifts everything)
l.push_front(44);                // O(1)
f.push_front(44);                // O(1)

int len_v = v.size();            // O(1)
int len_l = l.size();            // O(1)
// f.size() doesn't exist; forward_list doesn't track length
```

Operation	`std::vector`	`std::list`	`std::forward_list`
Append at end	$O(1)$ amortized	$O(1)$	no direct op
Prepend at front	$O(n)$	$O(1)$	$O(1)$
Insert after a known position	$O(n)$	$O(1)$	$O(1)$
Indexed access `v[i]`	$O(1)$	$O(i)$	$O(i)$
Search by value	$O(n)$	$O(n)$	$O(n)$
`size()`	$O(1)$	$O(1)$	not provided

Gotcha Pick the implementation from the *access pattern*

"I need a list" doesn't tell you which container to use. Ask:

- Will I mostly `push_back`? `vector`.
- Will I insert and remove in the middle a lot? `list`.
- Do I need indexed random access? `vector`.
- Do I need to splice or move sublists around? `list`.

For 80% of everyday code, `std::vector` wins even when it looks like the "wrong" choice, because its cache behavior and constant-factor speed beat the linked-list options for moderate n.

1.5 What this chapter builds

Key idea The plan

- **Singly linked list.** Nodes with one `next` pointer. Implements the list ADT with $O(1)$ insert at known positions but no backward traversal.
- **Doubly linked list.** Nodes with `next` and `prev` pointers. Enables $O(1)$ removal given a node pointer, and efficient reverse iteration.
- **Array lists (dynamic arrays).** Contiguous storage, $O(1)$ indexed access, amortized $O(1)$ `push_back`.
- **Stacks and queues.** Specializations of the above.

By the end of the chapter, every row in the C++ table above will be something you can explain, not just memorize.

Aside The pattern we'll repeat from now on

For every ADT in this chapter and the next, we'll do:

1. Define the ADT (today).
2. Implement it with one or two structures.
3. Analyze each operation's cost.
4. Compare to the alternatives.

You saw this cycle informally for the sorts in Chapter 3. From here on, it's the standard structure of every chapter.

2 4.2 Singly Linked Lists: Append and Prepend

This is the first concrete implementation of the list ADT: the **singly linked list**. Instead of storing items in a contiguous block of memory (like an array), a linked list scatters them across individually-allocated **nodes** that "point" to each other. The payoff is cheap insertion and deletion at known positions; the price is slower traversal and higher per-item memory overhead.

Definition — Singly linked list

A **singly linked list** is a sequence of nodes, where each **node** stores a data value and a `next` pointer to the following node. The list itself holds a **head** pointer (to the first node) and usually a **tail** pointer (to the last node). The last node's `next` is `nullptr`.

Key idea Picture the list

`head` $\rightarrow$ `[7 | •]` $\rightarrow$ `[9 | •]` $\rightarrow$ `[5 | null]` $\leftarrow$ `tail`

Each bracketed cell is a node. The left half is the data; the right half is the `next` pointer. `head` points to the first cell; `tail` points to the last; the last cell's `next` is `nullptr`.

2.1 The node type in C++

Example A node and a list wrapper

```
struct Node {
    int   data;
    Node* next;

    Node(int d) : data(d), next(nullptr) {}
};

struct LinkedList {
    Node* head = nullptr;
    Node* tail = nullptr;
};
```

head is how the list knows where to start; `tail` is the optimization that lets append run in $O(1)$ without walking the list. An empty list has both set to `nullptr`.

Gotcha Why `tail` is load-bearing

Without a `tail` pointer, appending to a linked list would require walking from head to the end – $O(n)$ per append, $O(n^2)$ to build the list. *Always* cache the tail when you care about append speed.

2.2 Append: add at the end

Two cases: an empty list needs its head and `tail` both pointed at the new node; a non-empty list only has to reach through the existing tail.

Example append

```
void append(LinkedList& list, Node* newNode) {
    if (list.head == nullptr) {          // empty list
        list.head = newNode;
        list.tail = newNode;
    } else {                              // non-empty list
        list.tail->next = newNode;        // stitch on the end
        list.tail       = newNode;        // advance tail
    }
}
```

Key idea Why append is $O(1)$

The work is constant: check one pointer, write two pointers. No scan of the list. The pre-existing newNode→`next` is assumed to already be `nullptr` from its constructor; if you're reusing a node, set it explicitly.

Gotcha The empty-list branch is not optional

If you forget the empty-list check and write `list.tail→next = newNode` when `tail` is `nullptr`, you get a null-pointer dereference and a crash. Every linked-list operation that touches `head` or `tail` needs to handle the empty case explicitly. This is the single most common linked-list bug.

2.3 Prepend: add at the front

Symmetric to append, but we touch `head` instead of `tail`.

Example prepend

```
void prepend(LinkedList& list, Node* newNode) {
    if (list.head == nullptr) {          // empty list
        list.head = newNode;
        list.tail = newNode;
    } else {                              // non-empty list
        newNode->next = list.head;        // point new node at old head
        list.head     = newNode;          // new node is the new head
    }
}
```

Key idea The order of the two assignments matters

In the non-empty branch, `newNode→next = list.head` must come *before* `list.head = newNode`. Swap them and you overwrite `head` with `newNode` first – then the subsequent `newNode→next = list.head` makes the node point at itself, losing the rest of the list. This is the classic pointer-update gotcha: **always copy out before you overwrite**.

Aside Why prepending into an array is different

On an array-backed list, prepending means shifting every existing item one slot to the right: $O(n)$. On a linked list it's just two pointer writes: $O(1)$. The linked list doesn't have to keep items adjacent in memory, so it never needs to shift.

2.4 Cost summary

Operation	Singly linked list	Why
`append`	$O(1)$ with tail pointer	Two writes; no traversal
`prepend`	$O(1)$	Two writes; no traversal

Both basic insertions are constant time. That's the primary advantage of a linked list over an array for workloads that repeatedly add at the ends.

2.5 Language conventions for "null"

Definition — Null

A **null pointer** is a pointer that points to nothing. Every pointer must either point to a valid object or be explicitly null. Languages name this differently:

- C++: `nullptr` (modern), `NULL` (old C-style macro, avoid)
- C: `NULL`
- Java: `null`
- Python: `None`
- Rust / Swift: they avoid null via the `Option` type

In all cases the concept is the same: a sentinel value indicating "no object here."

Gotcha Use `nullptr`, not `NULL` or `0`

In modern C++ (C++11 and later), always use `nullptr`. It's type-safe: `NULL` is often defined as plain `0`, which can accidentally be taken as an integer in overload resolution. `nullptr` has its own dedicated type (`nullptr_t`) and can't be confused with an integer. Every linked-list node in this course will initialize its pointers to `nullptr`.

2.6 What's coming next

Key idea The remaining SLL operations

Append and prepend are the easy cases because they touch the list's endpoints, where we already have pointers. The operations coming in 4.3 – insert-after-a-known-node, and remove-by-value – are harder because they involve traversing the list or keeping a *trailing* pointer to the node *before* the one you want to modify. That trailing-pointer pattern is the core awkwardness of singly linked lists and the main motivation for **doubly** linked lists a few sections later.

Aside C++ owns its memory explicitly

The pseudocode takes a `newNode` as a parameter and doesn't discuss where it came from. In C++, you'll usually `new Node(x)` to allocate and `delete` to free. Later, `std::list` does all of that inside its `push_back(x)`. This chapter works with raw pointers to expose the mechanics; in production code you'd use the STL container or smart pointers (`std::unique_ptr`) to automate the memory management.

3 4.3 Singly Linked Lists: Insert-After

Append and prepend covered the two endpoints. **Insert-after** is the general case: given a pointer to some existing node `curNode`, splice a `newNode` in right after it. The operation is still $O(1)$ – because we already have a pointer to the splice point – but the bookkeeping has to handle three cases instead of two.

Definition — Insert-after

InsertAfter(list, curNode, newNode) inserts `newNode` immediately after `curNode` in the list. If `curNode` is `null` (or the list is empty), `newNode` becomes the only node. If `curNode` is the current `tail`,

newNode becomes the new tail.

3.1 The three cases

The pseudocode looks like it's handling three unrelated scenarios, but they all flow from the same question: *which of the list's invariants does this insertion threaten?*

Key idea What the list has to stay true to

A well-formed singly linked list always satisfies:

1. head points to the first node or is null.
2. tail points to the last node or is null.
3. The last node's next is null.
4. head is null iff tail is null.

Every insertion must preserve all four. The three cases in the algorithm each exist because a different invariant is at risk.

Case	What needs updating
Empty list	Both head and tail
curNode is the tail	Tail's next pointer, and tail itself
curNode is interior	Just two next pointers (no head/tail changes)

3.2 The algorithm

Example insertAfter

```
void insertAfter(LinkedList& list, Node* curNode, Node* newNode) {
    if (list.head == nullptr) {                    // Case 1: empty list
        list.head = newNode;
        list.tail = newNode;
    }
    else if (curNode == list.tail) {                // Case 2: after tail
        list.tail->next = newNode;
        list.tail       = newNode;
    }
    else {                                           // Case 3: interior
        newNode->next = curNode->next;
        curNode->next = newNode;
    }
}
```

Gotcha Case 3 is where most beginners trip

The interior case has two assignments, and *order matters*:

1. newNode→next = curNode→next – make newNode point at what used to come after curNode. *Copy out before overwriting.*
2. curNode→next = newNode – then slot newNode in after curNode.

Swap them and you overwrite `curNode→next` first, losing the rest of the list forever. Same pointer-update discipline as in prepend – make sure the new node is fully stitched *before* you change the node already in the list.

Picture of the interior splice

Before:

```
... → [curNode | •] → [nextNode | ...] → ...
```

Step 1: `newNode→next = curNode→next`

```
... → [curNode | •] → [nextNode | ...] → ...
                                ↖
                      [newNode | •]
```

Step 2: `curNode→next = newNode`

```
... → [curNode | •] → [newNode | •] → [nextNode | ...] → ...
```

Two pointer writes, no scan. Constant time.

3.3 Why we need the curNode argument

Key idea "Insert at position k" isn't a list-ADT primitive

A client that wants to insert at a specific numeric position k has to *walk* the list to find the node at index $k-1$ first. That walk is $O(k)$, not $O(1)$. The list ADT's primitive is `InsertAfter(curNode, newNode)` – it assumes you already have a pointer to the right spot. How you got that pointer (cached from before, returned by a previous operation, walked to find) isn't the ADT's concern.
This is why singly linked lists are often described as supporting "$O(1)$ insertion *given a pointer*" – the pointer is assumed to be available.

3.4 Mapping to `std::list`

Example The STL version

```cpp
#include <list>
std::list<int> l = {7, 9, 5};

auto it = std::next(l.begin(), 1);   // iterator to the "9"
l.insert(it, 42);                    // inserts *before* it: 7, 42, 9, 5

auto it2 = std::next(l.begin(), 1);  // iterator to the "42"
l.insert(std::next(it2), 100);       // inserts before "9": 7, 42, 100, 9,
    5
```

`std::list::insert(pos, x)` inserts *before* pos. To insert *after* a position, pass `std::next(pos)`. The STL picks "before" because that's what `pos = end()` naturally means – insert at the end. Under the hood it's the same two-pointer splice, just with a doubly linked list.

Aside Why the pseudocode picks "after" instead of "before"

On a singly linked list, inserting *before* a node is surprisingly annoying – you need a pointer to the node *that points to* the target, i.e. its predecessor. The list doesn't provide that; you'd have to walk from `head` to find it, making the operation $O(n)$. Inserting *after* only needs the target itself. That's why the SLL ADT exposes `InsertAfter`, not `InsertBefore`. Doubly linked lists, with back-pointers, get both for free.

3.5 Cost

Operation	Singly linked list	Why
`insertAfter(curNode, ...)`	$O(1)$	Two pointer writes; no scan
Find `curNode` by value or index	$O(n)$	Walk from head
Insert by index k	$O(k) + O(1) = O(n)$	Walk first, then splice

Gotcha The $O(1)$ claim is only about the splice, not the find

Everyone who says "linked lists have $O(1)$ insertion" is technically right about the splice, but the *find* is $O(n)$. Unless you're iterating the list and splicing as you go, or you already have a cached node pointer, the practical cost of "insert an item at position k" is linear. This is the biggest source of confusion about linked-list performance.

4 4.4 Singly Linked Lists: Remove

Removal in a singly linked list has the same shape of problem as insertion: you need to update a `next` pointer that lives in the *predecessor* of the node you want to delete – and a singly linked list doesn't make that predecessor easy to reach. The workaround is the same one used by `insertAfter`: name the operation **RemoveAfter**, and make the caller pass the predecessor pointer.

Definition — RemoveAfter

RemoveAfter(list, curNode) removes the node *immediately after* `curNode`. Special case: if `curNode` is `null`, the function removes the *head* (the first node has no predecessor inside the list itself).

4.1 Why predecessor-based removal?

Key idea The asymmetry that makes this awkward

To remove node X from a linked list you have to rewrite `prev→next` to skip over X – but in a singly linked list, `prev` isn't available from X itself; there's no back pointer. The list has three realistic options:

1. **Walk from head** to find `prev`. $O(n)$.
2. **Make the caller hand us `prev`** and expose `RemoveAfter(prev)`. $O(1)$.
3. **Switch to a doubly linked list** where each node knows its predecessor. $O(1)$, but more memory.

The textbook's ADT picks option 2. Option 3 is what we'll build in 4.5.

4.2 The three cases

Two variables carry the state of the splice:

- curNode – the node *before* the one being removed (or null if we're removing the head).
- sucNode – the node *after* the one being removed (i.e., the new successor that the splice will connect to).

Case	What gets updated
Remove the head (curNode null)	head moves to successor; if that's null, also null out tail.
Remove an interior node	curNode→next moves to successor. No endpoint changes.
Remove the tail (curNode→next→next null)	curNode→next becomes null; also move tail to curNode.

4.3 The algorithm

Example removeAfter

```
void removeAfter(LinkedList& list, Node* curNode) {
    // Case A: remove the head
    if (curNode == nullptr && list.head != nullptr) {
        Node* toFree = list.head;
        Node* sucNode = list.head->next;
        list.head = sucNode;
        if (sucNode == nullptr) {        // list is now empty
            list.tail = nullptr;
        }
        delete toFree;
        return;
    }

    // Case B / C: remove the node after curNode (if one exists)
    if (curNode != nullptr && curNode->next != nullptr) {
        Node* toFree = curNode->next;
        Node* sucNode = curNode->next->next;
        curNode->next = sucNode;
        if (sucNode == nullptr) {        // we just removed the tail
            list.tail = curNode;
        }
        delete toFree;
    }
}
```

The textbook's pseudocode omits the delete. In real C++ you *must* free the removed node, or you leak memory. Python, Java, and JavaScript's garbage collectors would do it for you; C++ makes you explicit.

Gotcha Forgetting delete is a memory leak

Every new needs a matching delete unless a smart pointer is handling the ownership. Hand-rolled linked lists are one of the easiest places to leak memory in C++ because the node you unlink is no longer reachable from head, so nothing else will ever free it. **Cache the node before unlinking, reassign the pointers, then delete the saved node.**

Gotcha Do not `delete` before you finish reading

`Node* sucNode = curNode→next→next;` reads through `curNode→next`. If you `delete` `curNode→next` first, the subsequent dereference is undefined behavior. Order of operations:

1. Cache the node to be deleted.
2. Cache any pointers you'll need *from* it.
3. Rewrite the surrounding pointers.
4. `delete` the cached node.

This "read-then-rewrite-then-delete" rhythm is the heart of safe pointer surgery.

Picture of the interior splice

Before:

```
... → [curNode | •] → [victim | •] → [sucNode | ...] → ...
```

After `curNode→next = sucNode`:

```
... → [curNode | •] → [sucNode | ...] → ...
```

The `victim` node is unreachable – `delete` it.

4.4 Cost

Operation	Singly linked list	Why
`removeAfter(curNode)`	$O(1)$	Two pointer writes; no scan
Remove by value (find + remove)	$O(n)$	Linear walk to find the predecessor
Remove at index k	$O(k) = O(n)$	Walk to position, then splice

4.5 Why we don't have `Remove(node)` directly

Key idea The "remove given just the victim" trick

There's a famous trick interview question: "Given only a pointer to the node you want to delete in a singly linked list (not the head), delete it without a predecessor pointer."
The trick: *copy the next node's data into the victim and delete the next node instead.* Works for interior nodes; fails for the tail (there's no successor to pluck data from).
Real libraries avoid this contortion because it has subtle issues: dangling references held by callers, data-copy cost if values are large, ordering effects on equal-valued neighbors. *Doubly linked lists* solve the problem cleanly – they give you the predecessor pointer for free.

4.6 All five SLL operations together

Now that we have append / prepend / insert-after / remove-after, we can fill in the canonical cost table for a singly linked list with both head and tail pointers:

Operation	Cost	Assumption
`append(x)`	$O(1)$	Tail pointer cached
`prepend(x)`	$O(1)$	–
`insertAfter(p, x)`	$O(1)$	Caller has pointer `p`
`removeAfter(p)`	$O(1)$	Caller has pointer `p`
`search(x)`	$O(n)$	Linear scan
`at(i)`	$O(i)$	Walk from head
`remove(x)` by value	$O(n)$	Search + splice

Aside `std::forward_list` is this structure in the STL

`std::forward_list` is the C++ singly linked list. Its iterators support `insert_after` and `erase_after` for exactly the reason described here – those are the $O(1)$ operations the structure can actually give you. It deliberately omits `size()`, `push_back`, and bidirectional iteration because giving them would either force $O(n)$ costs or require back-pointers (which would make it `std::list`).

Gotcha Linked lists rarely win in modern C++

Despite the clean $O(1)$ insertion/deletion, singly linked lists lose to `std::vector` on most real workloads – even ones that intuitively seem to favor linked structure. The reasons are cache locality (a vector is one contiguous block; linked nodes are scattered) and per-node overhead (one pointer per item in an SLL, two in a DLL, zero in a vector). Use a linked list when you *need* stable pointers into the middle of the list or when you splice big ranges; otherwise reach for a vector.

5 4.5 Linked List Search

Search on a linked list is exactly what you'd expect: start at the head, walk node-by-node following `next` pointers, stop when you find a match or hit `null`. This is *linear search* from Chapter 3, applied to a linked-list structure. It's worth its own section because linked-list search exposes a recurring pattern – the *walker idiom* – that will be the scaffolding for traversal, reversal, length computation, and many other linked-list algorithms.

Definition — Linked-list search

Given a list and a **key**, search walks from the head and returns the first node whose `data` equals the key. If no node matches, it returns `nullptr`.

5.1 The algorithm

Example search

```
Node* search(const LinkedList& list, int key) {
    Node* curNode = list.head;
    while (curNode != nullptr) {
        if (curNode->data == key) {
            return curNode;
        }
        curNode = curNode->next;
```

```
    }
    return nullptr;
}
```

Three things to notice: the loop advances through `curNode = curNode→next`; the termination check is on the pointer, not the data; and we return the *pointer to the node*, not its index.

Key idea The walker idiom

The structure `Node* curNode = list.head; while (curNode≠nullptr) { ...; curNode = curNode→next; }` is the linked-list equivalent of `for (i = 0; i < n; ++i)` on an array. Almost every linked-list algorithm in this chapter – print, length, reverse, remove-by-value, insert-sorted – is built on this idiom. Memorize it.

5.2 Why return a pointer instead of an index

Gotcha Indices aren't the right return type for linked lists

On a vector, `find` returns an index. On a linked list, returning an index would be almost useless – the caller would have to walk from head *again* to reach that position. Returning a *pointer to the node* gives the caller immediate access to the data and to its neighbors for subsequent `InsertAfter` or `RemoveAfter` calls. The ADT is designed around passing these node pointers around.
The STL mirrors this: `std::find` on a vector returns an iterator that's effectively a pointer into the array; `std::find` on a list returns an iterator that wraps a node pointer. Same *idea*, different underlying representation – which is exactly what an ADT/iterator abstraction is for.

5.3 Cost

Key idea $O(n)$, always

Linked-list search has to examine each node in turn until it finds the key or exhausts the list. No structural shortcut is available – we can't binary-search a linked list, because reaching the middle is $O(n/2) = O(n)$ on its own. Sorted or unsorted, the linked-list search runtime is $\Theta(n)$ worst case.

Case	Comparisons	Runtime
Key at head	1	$O(1)$
Key at position k	$k+1$	$O(k)$
Key absent	n	$O(n)$

Gotcha Sorting a linked list doesn't help search

On an array, sorting unlocks binary search: $O(\log n)$. On a linked list, it does not. Even with a sorted linked list, you can't jump to the middle element in constant time. The best you can do is stop early when a value exceeds the key ("early exit" on sorted data):

```
while (curNode != nullptr && curNode->data < key) {
    curNode = curNode->next;
}
return (curNode && curNode->data == key) ? curNode : nullptr;
```

Helpful on hits near the front; still $\Theta(n)$ worst case.

5.4 Linked-list search is hostile to CPU caches

Key idea Why array search beats linked search in practice

Asymptotically, linear scan on a vector and linear search on a linked list are both $\Theta(n)$. On real hardware they differ by $10\times$ or more. The reason: a vector stores values in a single contiguous block of memory, so the CPU's prefetcher can pull ahead and cache the next dozens of elements before you ask for them. A linked list's nodes can be scattered anywhere in the heap, and following `next` pointers is a series of dependent loads that the prefetcher cannot anticipate. Every `curNode = curNode→next` potentially stalls waiting on main memory.

This is the top reason modern code prefers `std::vector` even when textbook analysis suggests `std::list` would work.

5.5 Two common variations

Example Return a pointer *and* its predecessor

Sometimes a caller wants both the found node and the one before it – so they can immediately `RemoveAfter`. The idiom tracks `prev` alongside `curNode`:

```
std::pair<Node*, Node*> searchWithPrev(const LinkedList& list, int key) {
    Node* prev = nullptr;
    Node* cur  = list.head;
    while (cur != nullptr) {
        if (cur->data == key) return {cur, prev};
        prev = cur;
        cur  = cur->next;
    }
    return {nullptr, nullptr};
}
```

The `prev` pointer is the "trailing pointer" trick. It's the closest a singly linked list gets to a doubly linked list's back pointer, synthesized fresh on each walk.

Example Recursive search

```
Node* searchRec(Node* cur, int key) {
    if (cur == nullptr)        return nullptr;
    if (cur->data == key)      return cur;
    return searchRec(cur->next, key);
}
```

Same algorithm, recursive shape. Intellectually clean, but each call adds a stack frame – $O(n)$ recursion depth. The iterative version uses $O(1)$ stack. Prefer iterative for long lists; recursive is just for illustration or when the language (e.g. functional languages with tail-call optimization) supports it

cheaply.

Gotcha Linked lists and recursion don't mix well in C++

A 100,000-node list processed with recursive `search` will happily blow your stack. Tail-call elimination isn't guaranteed in C++, and even when compilers do it, they don't always. Keep the iterative version as your default for any traversal that could hit long lists.

5.6 What search gives you

Key idea The building block

`search` is the first operation that doesn't already have a cached pointer handed to it. Most higher-level operations you'd write – "remove all nodes with value x," "insert y after the first x," "count how many x's are in the list" – are variations of search with different actions on match. The walker idiom plus a small change at the match site gives you almost everything.

6 4.6 Doubly Linked Lists

The pain of a singly linked list lives at the `prev` pointer it doesn't have: no cheap reverse traversal, no removal without a predecessor argument, no insertion "before" a known node. The **doubly linked list** spends one extra pointer per node to erase all of these problems. You get bidirectional traversal, $O(1)$ removal given the victim itself, and a cleaner (if wordier) splice operation.

Definition — Doubly linked list

A **doubly linked list (DLL)** is a linked list where each node has two pointers: `next` to its successor and `prev` to its predecessor. The list still holds `head` and `tail` pointers; the head node's `prev` is `null` and the tail node's `next` is `null`.

Key idea The picture

`null` ← [• | 7 | •] ↔ [• | 9 | •] ↔ [• | 5 | •] → `null`

Each cell has `prev` on the left and `next` on the right. The bidirectional arrows capture that every link is traversable both ways.

6.1 The node type in C++

Example A DLL node

```
struct DNode {
    int    data;
    DNode* next;
    DNode* prev;

    DNode(int d) : data(d), next(nullptr), prev(nullptr) {}
};

struct DLinkedList {
```

```
    DNode* head = nullptr;
    DNode* tail = nullptr;
};
```

One extra pointer per node compared to an SLL. On a 64-bit system that's 8 extra bytes per item; for 32-bit `int` data plus two 64-bit pointers, a DLL node is already 20 bytes of pointers and padding wrapping 4 bytes of data – a 5× memory overhead versus a `std::vector<int>`. Bring this up when someone claims linked lists are "cheap."

6.2 Invariants

Key idea What a well-formed DLL always satisfies

1. If `node→next` is not null, `node→next→prev` equals `node`.
2. If `node→prev` is not null, `node→prev→next` equals `node`.
3. `head→prev == nullptr`, `tail→next == nullptr`.
4. `head == nullptr` iff `tail == nullptr`.

Every operation below has to restore all four invariants. A DLL bug is almost always a forgotten back-pointer update.

6.3 Append

Example append on a DLL

```
void append(DLinkedList& list, DNode* newNode) {
    if (list.head == nullptr) {          // empty list
        list.head = newNode;
        list.tail = newNode;
    } else {                              // non-empty list
        list.tail->next = newNode;        // stitch forward
        newNode->prev   = list.tail;      // stitch backward
        list.tail       = newNode;        // advance tail
    }
}
```

Compared to the SLL version, we do *one extra write*: set `newNode→prev = list.tail` before moving the tail. Still $O(1)$.

Gotcha Every next-write deserves a twin prev-write

The rule of thumb when implementing any DLL operation: for every `a→next = b` you write, ask "did I also write `b→prev = a`?" Skip that check and you'll silently break invariant 1. Many DLL bugs only show up when iterating the list *backwards* – which can be long after the broken write.

6.4 Prepend

Symmetric to append. Every `tail` becomes `head`; every `next` swaps with `prev`.

Example prepend on a DLL

```
void prepend(DLinkedList& list, DNode* newNode) {
    if (list.head == nullptr) {          // empty list
        list.head = newNode;
        list.tail = newNode;
    } else {                              // non-empty list
        newNode->next  = list.head;       // forward link in
        list.head->prev = newNode;        // backward link in
        list.head       = newNode;        // advance head
    }
}
```

6.5 What the back-pointer unlocks

Key idea Why the extra pointer is worth it

- **Reverse iteration.** Walk from `tail` following `prev`. SLL can't do this without reversing the list or recursing.
- **`Remove(node)` with just the victim.** Given a pointer to any node, you can unlink it without having the predecessor cached – you read `node→prev` to find it. SLL requires `removeAfter(prev)`.
- **`InsertBefore(node, x)`.** Same reason: `node→prev` gives you everything needed to splice before.
- **`PrintReverse`, `Sort` with bidirectional access, iterators in both directions.**

Every common list operation is either equivalent in cost to SLL or strictly easier on a DLL.

6.6 When the extra pointer is *not* worth it

Gotcha Two places SLL still wins

- **Memory-sensitive scenarios.** One pointer per node instead of two is 50% less pointer overhead. If you're storing millions of tiny items, this actually matters. `std::forward_list` exists precisely for this.
- **Intrusive linked lists in kernels or embedded systems.** Fewer pointers mean smaller per-object headers.

Outside those niches, the DLL wins on ergonomics. `std::list` is a DLL, not an SLL, for this reason.

6.7 Cost of the endpoint operations

Operation	SLL	DLL	Notes
`append`	$O(1)$	$O(1)$	One extra write on DLL
`prepend`	$O(1)$	$O(1)$	One extra write on DLL
`insertAfter(n)`	$O(1)$	$O(1)$	DLL also has `insertBefore`
`remove(n)`	needs prev, so $O(n)$ to find it, then $O(1)$	$O(1)$ given node	DLL wins outright

6.8 `std::list` is a circular DLL

Aside How the STL implements DLL

`std::list` uses a circular doubly linked list with a *sentinel node*. A dedicated sentinel (not real data) sits between the tail and the head; the list is never empty in the linked-structure sense – even an empty list has the sentinel pointing at itself. This trick eliminates all the "is the list empty?" special cases and gives `end()` a stable iterator to return, but you won't write it by hand in an intro course. The plain head/tail/null-terminator DLL here is the standard pedagogy.

6.9 Where this is going

Key idea The rest of the DLL arc

Appending and prepending are the warm-up. The interesting DLL operations come next:

- **InsertAfter** (4.7 if the textbook follows the pattern): same as SLL plus one back-pointer update per side.
- **Remove(node)**: the big payoff – no `prev`-arg needed.
- **PrintReverse**: trivial on DLL, annoying on SLL.

Once you've seen DLL's remove, the trade-off between SLL and DLL should be obvious: you pay one extra pointer per node for a much cleaner API. For 99% of production code, you take the deal.

Gotcha C++ memory hygiene reminder

The same `delete` discipline from the SLL remove section applies here: save the victim pointer, unlink all four neighboring pointers (two `next`, two `prev`), `delete` the victim. Smart pointers (`std::unique_ptr`) complicate DLL ownership because both `next` and `prev` "own" each other – don't naively wrap pointers in `unique_ptr` on both sides or you'll get cyclic ownership. Typical practice: `prev` is a raw pointer; `next` is a `unique_ptr`. Or, skip this and just use `std::list`.

7 4.7 Doubly Linked Lists: Insert-After

Insert-after on a doubly linked list is the same algorithm as the SLL version from 4.3, with one extra pointer write per affected side to keep the `prev` links in sync. The interior case – where all four invariants meet – requires **four** pointer updates instead of the SLL's two. Each of the four has to happen in the right order, and every pattern you pick here will be the template for the rest of the DLL operations.

Definition — InsertAfter on a DLL

InsertAfter(list, curNode, newNode) inserts `newNode` immediately after `curNode`. As in the SLL version, the three cases are "empty list," "`curNode` is the tail," and "interior." Each case preserves the DLL's four invariants (both directions linked, head/tail nulls, empty-list agreement).

7.1 The three cases

Case	Pointer writes	Structures affected
Empty list	2	head, tail
curNode is tail	3	tail's next, new node's prev, tail
Interior	4	curNode's next, sucNode's prev, new node's next and prev

7.2 The algorithm

Example insertAfter on a DLL

```
void insertAfter(DLinkedList& list, DNode* curNode, DNode* newNode) {
    if (list.head == nullptr) {                          // Case A: empty
        list.head = newNode;
        list.tail = newNode;
    }
    else if (curNode == list.tail) {                      // Case B: after
        tail
        list.tail->next = newNode;
        newNode->prev   = list.tail;
        list.tail       = newNode;
    }
    else {                                                // Case C: interior
        DNode* sucNode = curNode->next;
        newNode->next  = sucNode;
        newNode->prev  = curNode;
        curNode->next  = newNode;
        sucNode->prev  = newNode;
    }
}
```

Case C, step by step

The interior case is four pointer writes. The order is *flexible*, but the intent must stay consistent: **wire the new node first, then redirect the existing ones**. Writing the old pointers first while the new node is still dangling breaks invariants mid-flight and can get you into trouble on multi-threaded or signal-interrupted code. Safe ordering:

Step	Write
1	newNode→next = curNode→next (remember the successor first)
2	newNode→prev = curNode (wire the predecessor)
3	curNode→next = newNode (splice forward)
4	sucNode→prev = newNode (splice backward)

Gotcha Don't overwrite curNode→next before caching it

The sucNode local variable exists precisely to cache curNode→next *before* we overwrite it in step 3. If you skip the cache – write step 3 first and then try to reach sucNode via curNode→next – you've lost

the successor. Classic prerequisite for a "why are half my nodes missing" debugging session. **Cache out before you overwrite.**

7.3 Picture of the interior splice

Before:

`... ↔ [curNode] ↔ [sucNode] ↔ ...`

After:

`... ↔ [curNode] ↔ [newNode] ↔ [sucNode] ↔ ...`

Four arrows redirected. Nothing else changes – the head, the tail, and any nodes further away are untouched.

7.4 Why inserting *before* is now trivial

Key idea InsertBefore for free

On a singly linked list, `InsertBefore(node, x)` requires finding the predecessor – a walk from `head`. On a doubly linked list, `node→prev` is right there. `InsertBefore(node, x)` is `InsertAfter(node→prev, x)` with one special case for `node == head`. That's why `std::list` and almost all modern container APIs use "insert *before* position" as their primitive: on a DLL it's natural.

Example `insertBefore` on a DLL

```
void insertBefore(DLinkedList& list, DNode* curNode, DNode* newNode) {
    if (list.head == nullptr) {
        list.head = newNode;
        list.tail = newNode;
    }
    else if (curNode == list.head) {                    // before the head
        newNode->next   = list.head;
        list.head->prev = newNode;
        list.head       = newNode;
    }
    else {
        DNode* predNode = curNode->prev;
        newNode->next  = curNode;
        newNode->prev  = predNode;
        predNode->next = newNode;
        curNode->prev  = newNode;
    }
}
```

Symmetric to `insertAfter`, mirror-imaged. Same structure, same four-writes-in-interior-case, same cache-predecessor-first discipline.

7.5 Cost

Operation	Cost	Assumption
`insertAfter(curNode, x)`	$O(1)$	Caller has `curNode`
`insertBefore(curNode, x)`	$O(1)$	Caller has `curNode`
`insert at index` k	$O(k)$	Walk from head, then splice

Gotcha "Four writes" vs. SLL's "two" is not a real disadvantage

A DLL insertion does $2\times$ the pointer writes of an SLL insertion, but both are constant-time. On modern hardware the extra writes are lost in the noise (pipelined, same cache line). What matters is that DLL inserts still don't scan or shift. You trade a few nanoseconds per insert for a vastly more useful API – cheap reverse iteration, cheap removal from an arbitrary node, cheap insertBefore.

7.6 In the STL

Example `std::list::insert`

```
std::list<int> l = {7, 9, 5};
auto it = std::next(l.begin(), 1);  // iterator to "9"

l.insert(it, 42);                    // 7, 42, 9, 5  (insert before)
l.insert(std::next(it), 100);        // 7, 42, 9, 100, 5 (insert after
    "9")
```

`std::list::insert(pos, x)` inserts *before* `pos`. This is the only primitive the STL provides – insert after is written as `insert(std::next(pos), x)`. The implementation under the hood is the four-pointer-write interior case above, wrapped in a circular-DLL-with-sentinel that also eliminates the "empty list" special case.

Key idea Where this is going

With insertAfter done, the remaining operation is **remove**. The payoff of the DLL structure shows up there most clearly: with `prev` pointers available, removal given just the victim takes constant time and doesn't need a `removeAfter` trick. Section 4.8 (or whatever number the textbook assigns it) will make that concrete.

8 4.8 Doubly Linked Lists: Remove

This is where the extra `prev` pointer pays its rent. **On a DLL, removing a node takes $O(1)$ given nothing but the node itself** – no predecessor argument, no backward scan. The SLL couldn't do this. The algorithm is also structurally cleaner: instead of three scenario branches, we use four *independent* checks that each handle one of the pointers that might need updating.

Definition — Remove on a DLL

Remove(list, curNode) unlinks `curNode` from the list and returns it (or frees it, depending on ownership). It updates up to four pointers: the predecessor's `next`, the successor's `prev`, and – if `curNode` was at an endpoint – the list's `head` or `tail`.

8.1 Four independent checks instead of three cases

The SLL's `RemoveAfter` had three cases. The DLL's `Remove` is organized differently: four independent "if this pointer exists, patch it" checks. Any combination of them fires depending on where the victim sits.

Condition	Action
`sucNode` $\neq$ `null`	`sucNode→prev = predNode`
`predNode` $\neq$ `null`	`predNode→next = sucNode`
`curNode == list.head`	`list.head = sucNode`
`curNode == list.tail`	`list.tail = predNode`

Key idea Why four conditions cover all scenarios

- Interior node: `sucNode` and `predNode` both exist; `curNode` is neither head nor tail. Only the first two conditions fire.
- Head removal, list has other nodes: `sucNode` exists, `predNode` is null, `curNode` is head. Conditions 1 and 3 fire.
- Tail removal, list has other nodes: `predNode` exists, `sucNode` is null, `curNode` is tail. Conditions 2 and 4 fire.
- Only node in the list: both `sucNode` and `predNode` are null; `curNode` is both head and tail. Only conditions 3 and 4 fire, setting both to `nullptr`. The list ends up empty.

Independence is elegant: each check restores one invariant, and the whole algorithm is just running all four.

8.2 The algorithm

Example `remove` on a DLL

```
void remove(DLinkedList& list, DNode* curNode) {
    DNode* sucNode  = curNode->next;
    DNode* predNode = curNode->prev;

    if (sucNode  != nullptr) sucNode->prev  = predNode;
    if (predNode != nullptr) predNode->next = sucNode;

    if (curNode == list.head) list.head = sucNode;
    if (curNode == list.tail) list.tail = predNode;

    delete curNode;
}
```

Six lines of logic, four conditional pointer writes, one `delete`. No traversal. $O(1)$ in all cases.

Gotcha Read `curNode→next` and `curNode→prev` *before* `deleteing`

As in the SLL case: cache the neighbors out of `curNode` first, *then* free the node. Reading through a freed pointer is undefined behavior. The order of operations in the code above is load neighbors → patch them → fix head/tail → `delete`. Follow that sequence and you'll be safe.

8.3 Picture

Before, for an interior removal:

`... ↔ [predNode] ↔ [curNode] ↔ [sucNode] ↔ ...`

After:

`... ↔ [predNode] ↔ [sucNode] ↔ ...`

The `curNode` is short-circuited by updating exactly two pointers: `predNode→next` and `sucNode→prev`. The victim is then unreachable from the list and can be safely freed.

8.4 The big comparison to SLL

This is the place where the SLL/DLL trade-off lands most clearly.

Operation	SLL	DLL
Remove, given node pointer	$O(n)$ (scan for prev) or use awkward `RemoveAfter(prev)`	$O(1)$ from the node
Remove by value	$O(n)$ (walk to find)	$O(n)$ (walk to find) + $O(1)$ unlink
Remove during a walk	$O(1)$ with trailing prev	$O(1)$ natively
Splice a node between two existing lists	painful; need prev in both places	trivial with the node's prev/next pointers

Key idea The "remove during a walk" idiom

On a DLL you can walk forward and decide-to-remove as you go without tracking a trailing pointer – `curNode→prev` gives it to you. This is why `std::list::remove_if` and related algorithms are so clean on `std::list`: remove wherever you like, the pointers are always available.

8.5 Cost

Operation	Cost	Assumption
`remove(node)`	$O(1)$	Caller has a pointer to the node
Remove by value	$O(n)$	Linear search to find, then $O(1)$ unlink
Remove at index k	$O(k)$	Walk to position, then $O(1)$ unlink
Remove all of value x	$O(n)$	Single walk; remove matches in place

8.6 Handling ownership in C++

Gotcha Who frees the node?

If `remove` calls `delete curNode`, the caller must not `delete` it again. But if you want the caller to take ownership (say, to reinsert the node into a different list), `remove` should return the pointer instead of deleting. A clean API picks one – mixing both leads to double-frees or leaks. `std::list::erase` deletes; `std::list::splice` moves. Pick the right primitive for the task.

Example Return-the-node variant

```
DNode* unlink(DLinkedList& list, DNode* curNode) {
    DNode* sucNode  = curNode->next;
```

```
    DNode* predNode = curNode->prev;
    if (sucNode  != nullptr) sucNode->prev  = predNode;
    if (predNode != nullptr) predNode->next = sucNode;
    if (curNode == list.head) list.head = sucNode;
    if (curNode == list.tail) list.tail = predNode;
    curNode->next = curNode->prev = nullptr;   // disconnect for safety
    return curNode;                            // caller owns it now
}
```

The `curNode→next = curNode→prev = nullptr` line matters: leaving stale pointers on a detached node invites bugs where the caller accidentally walks from it back into the old list.

8.7 Where this leaves the list ADT

Key idea The SLL/DLL decision, in one line

Singly linked list is the minimal implementation of the list ADT. *Doubly linked list* spends one extra pointer per node to make every operation that touches a known node constant time and every iteration bidirectional. For almost every real use case, take the deal.

Aside Where this is going

Chapter 4's next arc moves from generic lists to **restricted** list-based ADTs: stacks, queues, and deques. Each one is "a list with some operations removed so the remaining ones can be made cheaper or more predictable." Stacks and queues are both usually implemented as either a dynamic array or a (doubly) linked list; the work you've done here is the foundation for both.

9 4.9 Linked List Traversal

Traversal is the loop that ties linked lists back to every other container: "do something to each element." It is also the cheapest linked list operation to get right, because a linked list has exactly one shape of walk through it — `curNode = curNode->next`, repeated until the pointer falls off the end.

Definition — Traversal

An algorithm that visits every node in a list exactly once and performs some operation on each. The "operation" is a parameter of the traversal, not part of it: print, sum, check a predicate, copy into another container — the walk is the same.

The walker idiom, one more time

You have written this loop (or a close cousin) in every earlier section. It deserves its own name because it is the single building block under search, insert-after-value, remove-by-value, size, clear, copy, and almost any other "go look at everything" operation.

Example Forward traversal, the loop everyone writes

```
// Apply `visit` to every node in a singly-
// or doubly-linked list, front to back.
```

```
void traverse(Node* head) {
    for (Node* cur = head; cur != nullptr; cur = cur->next) {
        visit(cur);
    }
}
```

Three things make this loop airtight. First, the starting state is the list handle (`head`), not a guess — if the list is empty, `head` is `nullptr` and the loop body never runs. Second, the termination condition is `cur != nullptr`, which works for length 0, length 1, and length n with no special cases. Third, the advance step is a single field read; `next` is either a valid node or the sentinel `nullptr` that the list's append/prepend/remove code is contractually required to maintain.

Gotcha Never dereference cur inside the condition

A common rookie loop is `while (cur->next != nullptr)`, which walks *between* pairs of nodes. That form is useful for *predecessor-finding* (e.g., "find the node before the one to delete"), but it is the **wrong** loop for a traversal. Using it for traversal silently skips the tail node. Match the loop shape to the question you are answering.

Doubly-linked lists: reverse traversal for free

Because every node in a DLL carries a `prev` pointer and the list keeps a `tail` handle, you can walk the list backwards at the same cost as forwards. This is one of the concrete payoffs for the extra pointer per node discussed in section 4.6.

Example Reverse traversal on a DLL

```
void traverseReverse(DNode* tail) {
    for (DNode* cur = tail; cur != nullptr; cur = cur->prev) {
        visit(cur);
    }
}
```

A singly-linked list has no efficient reverse traversal. The only way to visit the nodes in reverse is to either reverse the list first (O(n) destructive), push every node onto a stack during a forward pass (O(n) space), or use recursion (which is push-onto- a-stack by another name, using the call stack). All three cost Theta(n) extra work or space on top of the walk itself. The DLL does it in zero extra work because the backward links were already built.

Cost

Operation	Cost	Notes
Forward traversal (SLL or DLL)	$\Theta(n)$	Must visit each node
Reverse traversal (DLL)	$\Theta(n)$	Symmetric to forward
Reverse traversal (SLL)	$\Theta(n)$	Plus $\Theta(n)$ space

The big-Theta is non-negotiable: to touch every element you pay n operations. The interesting cost is *per-step*, which is O(1) for both SLL and DLL because each step is one pointer indirection. This is what

makes the linked list traversal the same asymptotic cost as an array's `for (int i = 0; i < n; ++i)` even though the underlying machine work is quite different.

Aside Array traversal is faster in practice

A linked-list traversal is $\Theta(n)$, but so is an array's `for` loop — and on real hardware the array wins by a large constant factor. Array elements are contiguous, so the CPU prefetches the next cache line while the current one is being processed; linked-list nodes are scattered across the heap, so each `cur = cur->next` is likely to miss the cache and stall. Big-O says they're equal. The wall clock says otherwise. This is one reason `std::vector` is the default C++ container even when the textbook problem "wants" a list.

Traversal as the hidden primitive

Once you have the walker loop, almost every other "touch everything" operation is a specialization of it.

Operation	What the walker body does
`size()`	Increment a counter
`print()`	Print `cur->data`
`search(x)`	Compare, return if found
`sum()`	Accumulate `cur->data`
`clear()`	`delete prev`, advance
`copy()`	Append `cur->data` to new list
`contains(x)`	Compare, return `bool`

The reason linked list code looks repetitive across the chapter is that it *is* repetitive: every one of these operations is the same loop with a different body. If you find yourself writing the traversal scaffold for the fifth time in one class, that is the signal to factor out a higher-order helper that takes a callable — which is exactly what the STL does in `std::for_each`, `std::find_if`, and `std::accumulate`.

Example STL: traversal without writing the loop

```
#include <list>
#include <algorithm>
#include <numeric>
#include <iostream>

std::list<int> xs = {3, 1, 4, 1, 5, 9};

// print each
std::for_each(xs.begin(), xs.end(),
              [](int v) { std::cout << v << ' '; });

// find first element > 4
auto it = std::find_if(xs.begin(), xs.end(),
                       [](int v) { return v > 4; });

// sum
int total = std::accumulate(xs.begin(), xs.end(), 0);
```

`std::list` exposes iterators (`begin()`, `end()`, `rbegin()`, `rend()`) that are the library's name for "the current

walker." Advancing an iterator with `++it` is exactly the `cur = cur->next` step; the iterator just hides the pointer. Once you understand the raw loop, the STL version stops looking magical.

Gotcha Do not modify the list *during* a traversal

If the visit step can delete or re-link the current node, the naive loop is broken — you will read `cur->next` from a node you just `delete`'d. The fix is the cache-before- overwrite pattern from the remove sections: grab `Node* nxt = cur->next;` before doing anything destructive, then advance to `nxt` at the end of the loop body. The STL's erase-returns-the-next-iterator idiom (`it = xs.erase(it)`) is the same pattern with iterator dress.

Key idea Why this is the simplest section in the chapter

Every earlier linked-list operation had edge cases: empty list, single node, head, tail, interior, value-not-present. Traversal has **none** of those. The loop works for length 0 because `head == nullptr`. It works for length 1 because the single node's `next` is `nullptr`. It works for length n for the same reason. That simplicity is why every other linked-list algorithm is built on top of it — get the walker right once and reuse it forever.

10 4.10 Sorting Linked Lists

Sorting is the first algorithm in this chapter that isn't just a walker variant. It's also where the cost difference between arrays and linked lists stops being theoretical — many classic array sorts straight up don't work on a linked list, and the ones that do have a different implementation on each list flavor. Insertion sort was first analyzed on arrays in §3.15; here we adapt it to a linked-list traversal.

Key idea The core question

Which sorting algorithms transplant cleanly to a linked list, and which need random access so badly they fall apart without it? The answer splits neatly: algorithms that walk the data once or twice (insertion, merge) transfer; algorithms that jump around by index (quick, heap, shell) do not.

Insertion sort on a DLL: the easy case

Insertion sort on a doubly-linked list is almost identical to the array version. The outer loop visits each node starting from the second; the inner loop walks `prev` pointers back through the sorted prefix until it finds the right slot, then splices the current node there.

Example DLL insertion sort

```
void insertionSort(DList& L) {
    DNode* cur = L.head ? L.head->next : nullptr;
    while (cur) {
        DNode* nxt = cur->next;   // cache before mutation
        DNode* s   = cur->prev;
        while (s && s->data > cur->data) s = s->prev;

        L.remove(cur);
        if (!s) L.prepend(cur);
        else    L.insertAfter(s, cur);
```

```
        cur = nxt;
    }
}
```

The cache-`nxt`-before-mutating pattern is the same one you saw in remove-during-traversal (section 4.9). You can't advance via `cur->next` after `L.remove(cur)` splices `cur` out — its `next` may now point somewhere else, or may be stale.

Gotcha SLL insertion sort is structurally different

An SLL has no `prev` link, so you *cannot* walk backward from `cur` through the sorted prefix. The SLL version flips the search: walk from `head` forward until you find the insertion point, then splice. Same O(n^2) asymptotic, different loop structure. Don't assume a doubly-linked algorithm ports to singly-linked; check that every pointer read you need actually exists.

Example SLL insertion sort, forward-search variant

```
// Find the node *after which* `value` should go,
// or nullptr if it belongs at the head.
Node* findInsertPos(List& L, int value) {
    Node* before = nullptr;
    Node* cur    = L.head;
    while (cur && value > cur->data) {
        before = cur;
        cur    = cur->next;
    }
    return before;
}
```

Cost

Variant	Best	Average/Worst	Why
DLL insertion sort	$O(n)$	$O(n^2)$	Already-sorted skips inner loop
SLL insertion sort	$O(n)$	$O(n^2)$	Descending list = trivial

The best-case $O(n)$ trigger differs between the two: a sorted DLL never enters the inner loop (the `prev` data is already smaller), while a reverse-sorted SLL inserts every element at the head — `findInsertPos` returns `nullptr` immediately, which is also a no-op.

Which array sorts survive the linked-list trip?

Algorithm	Port?	Reason
Insertion sort	Yes	Only needs neighbor access
Merge sort	Yes	Splitting & merging is pointer surgery
Selection sort	Yes (ish)	Needs n linear "find min" passes
Shell sort	No	Gap-jumps require indexed access
Quicksort	Poor	Partition needs both-ends access
Heap sort	No	Heap needs parent/child by index

Key idea Why merge sort is the linked list sort

Merge sort is the textbook recommendation for linked lists. Splitting the list requires one walk to find the middle (`O(n)`, fine). Merging two sorted lists is pure pointer splicing: no array index math, no auxiliary buffer, no random access. Total cost: $O(n \log n)$ time, $O(\log n)$ stack, $O(1)$ extra heap. That is the *same* asymptotic as array merge sort *without* the $O(n)$ scratch array. Linked lists don't win many contests, but they win this one.

Aside What the STL actually does

`std::list::sort` is merge sort — specifically a bottom- up, iterative merge sort that avoids recursion overhead. `std::forward_list::sort` uses the same approach. `std::sort` on a `std::vector` is introsort (quick → heap fallback). The STL is explicit: you get different algorithms on different containers because the container shape determines what's cheap.

11 4.11 Sentinel (Dummy) Nodes

Every removal and insertion algorithm you've written so far has a prologue: a cluster of `if` statements for the empty list, the head, and the tail. Sentinel nodes — also called "dummy" nodes — are a structural trick that makes those special cases *disappear*. You pay one pointer of overhead and a pinch of conceptual weirdness; in return, every other method gets shorter.

Definition — Sentinel / dummy node

A permanent, data-less node that lives at the head (and optionally the tail) of a list. It is never returned to callers, never counts toward the list's size, and is never removed. Its purpose is purely structural: to guarantee that the "real" nodes always have a neighbor on one or both sides.

Why the head-and-tail cases exist

Take a close look at `ListRemove` from section 4.8: the very first lines check `if (curNode == list->head)` and `if (curNode == list->tail)`. Those branches exist because *head has no predecessor* and *tail has no successor*. Most of the body is just "update the list handle instead of an in-list pointer."

A sentinel makes head-has-no-predecessor false. Every real node now has a valid `prev` pointing to either another real node or the sentinel. The removal code never has to ask "am I at the head?" because it always has a predecessor to splice around.

Key idea The pattern

Special cases at the boundary usually mean the data structure is missing something at the boundary. Sentinels are the canonical fix: add a virtual neighbor so the boundary stops being a boundary. This same idea shows up again in trees (nil/leaf sentinels in red-black trees), in graph algorithms (super-source / super-sink nodes), and in strings (null terminators). Once you see it once you start seeing it everywhere.

One dummy vs. two dummies

Variant	Simplifies	Still special-cases
No sentinel	—	head, tail, empty
One head sentinel	head/empty	tail
Two sentinels	everything	none (for interior ops)

The two-sentinel DLL is the cleanest: `ListRemove` is just "unsplice," with no branches at all, because both neighbors are guaranteed to exist.

Example Two-sentinel DLL remove — branchless

```
void remove(DNode* cur) {
    // cur is a real node, so cur->prev and cur->next
    // are both non-null (they point at real nodes
    // or at the sentinels).
    cur->prev->next = cur->next;
    cur->next->prev = cur->prev;
    delete cur;
}
```

Compare that with the no-sentinel version's four pointer checks (section 4.8). Both are $O(1)$ — but the branchless version is faster in practice (the CPU loves straight-line code), shorter, and almost impossible to get wrong.

What sentinels cost

Cost	Amount	Notes
Extra memory	1 or 2 nodes	Constant, regardless of list size
Extra complexity	"size" is tracked separately	`size() != number of nodes`
Extra care	iterators must skip sentinels	Usually hidden in `begin()`/`end()`

The "size is tracked separately" point matters for correctness. A sentinel-based list must either keep a size counter or define "size" as "number of nodes minus the sentinels." Walking the whole list and counting no longer gives the right answer for a caller who expects 0 on an empty list.

Gotcha The empty-list invariant flips

Without sentinels, an empty list is `head == nullptr`. With two sentinels, an empty list has `head->next == tail` (or equivalently `tail->prev == head`). If you inherit sentinel code without noticing, an empty-list check like `if (L.head == nullptr)` silently becomes "is this list broken?" rather than "is

this list empty?" Update every such check to the new invariant when you add sentinels.

Do real C++ lists use sentinels?

Yes. `std::list` in libstdc++ and libc++ is a doubly-linked circular list with a single sentinel. The list object itself *contains* the sentinel inline (no heap allocation for an empty list), and `end()` returns an iterator to that sentinel. That's why dereferencing `end()` is undefined behavior but comparing against it is fine: the sentinel exists, it just doesn't hold a user value.

Aside Circular + sentinel = extremely clean

libstdc++'s `std::list` is circular: `tail->next` is the sentinel, `sentinel->next` is the head, and `head->prev` is the sentinel. With one sentinel you get two-sentinel semantics for free, because the sentinel is simultaneously "before head" and "after tail." If you're curious about the STL source, this is a good place to look — it's short, clever, and illustrative.

Key idea When to use sentinels in your own code

For one-off assignment code: probably don't bother — the branches you save aren't worth the conceptual weight. For library-quality data structures: almost always yes. The simplification to *every* algorithm (insert, remove, traverse, even iterators) more than pays back the one-time complexity of "there's a fake node at each end."

The payoff: `std::list::splice`

The circular-sentinel layout enables one operation that no contiguous container can match: moving an entire range of nodes between two lists in $O(1)$, regardless of range size. The STL exposes this as `std::list::splice`.

Example `std::list::splice`: the operation only a linked list can do in $O(1)$

```
#include <list>
std::list<int> a {1, 2, 3, 4, 5};
std::list<int> b {10, 20, 30};

// Move the [2, 3, 4] range from a into b before
// the position of 20. O(1) -- no copies, no allocation.
auto first = std::next(a.begin());      // points to 2
auto last  = std::next(a.begin(), 4);   // points to 5 (one past 4)
b.splice(std::next(b.begin()), a, first, last);

// a is now {1, 5}; b is now {10, 2, 3, 4, 20, 30}.
```

The cost is $O(1)$ because splice only rewires four pointers — the next/prev of the boundary nodes on each side. The range itself isn't touched. `std::vector` cannot do this without copying or moving every element in the range, an inherent $O(k)$ cost. This is the canonical answer to *"why would I ever use* `std::list` *when vector usually wins?"* If your workload splices large ranges between lists, the linked structure pays for its cache-hostility and per-node allocation overhead. Otherwise it usually doesn't.

12 4.12 Recursion on Linked Lists

A linked list is a recursive data structure in disguise: it is "a head node plus the rest of the list, which is also a linked list." That definition practically hands you the recursive algorithms for free. Every iterative walker from the last few sections has a recursive twin that is often shorter, and *occasionally* clearer — at a real performance cost.

Key idea The recursive mental model

Don't think "the function calls itself." Think "the function assumes its job on a smaller list is already done, and only has to handle one more node." If the list is empty, the work is zero. Otherwise, do the head, then trust the recursive call to handle the tail. That mindset makes the code write itself.

Forward traversal, recursive

Example Recursive forward traverse

```
void traverse(Node* node) {
    if (!node) return;        // empty list: done
    visit(node);              // head: do the work
    traverse(node->next);     // tail: trust recursion
}
```

Three lines, one branch. The base case is "empty list," which maps exactly to `node == nullptr`. The inductive step is "visit head, recurse on tail." This is the cleanest expression of a traversal you can write — and the iterative version from section 4.9 is still faster in practice.

Forward vs. reverse: one line swap

The striking thing about recursive list code is how little has to change to reverse the order. Swap "visit" and "recurse" and you get a reverse traversal:

Example Recursive reverse traverse

```
void traverseReverse(Node* node) {
    if (!node) return;
    traverseReverse(node->next);  // recurse FIRST
    visit(node);                  // then visit
}
```

This works on a *singly*-linked list — no `prev` pointer needed. The reversal is hiding in the call stack: each activation records a node, all activations unwind in LIFO order, and visits happen on the way out.

Key idea Recursion *is* a stack

The call stack is a stack. Recursion that does work "after the recursive call" is literally pushing things onto a stack and processing them in reverse. This is why reverse traversal on an SLL "costs nothing extra" recursively but $O(n)$ space iteratively — the cost didn't vanish, it's just hidden in the call stack.

Section 4.14 will make this explicit by building a stack ADT.

Recursive search

Example Recursive search

```
Node* search(Node* node, int key) {
    if (!node)              return nullptr; // not found
    if (node->data == key)  return node;    // found
    return search(node->next, key);         // keep looking
}
```

Three cases, each a single line. The third case is a *tail call*: the recursive call is the last thing the function does. Some compilers (including modern g++/clang with `-O2`) can recognize this and rewrite it as a loop, reclaiming the stack frame each step. With tail-call optimization, the recursive version has the same stack cost as the iterative one. Without it, a 10,000-element list will blow the default 8MB stack on Linux somewhere around recursion depth 100,000 — well above 10k, but closer than you think.

Cost: recursive vs iterative

Operation	Rec. time	Rec. space	Iterative space
Forward traverse	$\Theta(n)$	$\Theta(n)^*$	$\Theta(1)$
Reverse traverse	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$ (explicit stack)
Search	$\Theta(n)$	$\Theta(n)^*$	$\Theta(1)$

* With tail-call optimization, $\Theta(1)$. C++ does *not* guarantee TCO — treat it as a performance optimization, not a correctness property.

Gotcha Stack overflow is a real risk

A long linked list plus a recursive traversal plus a compiler that didn't eliminate the tail call = stack overflow. The default thread stack on Linux is 8MB; on Windows, 1MB; on embedded systems, often kilobytes. A stack overflow is usually a hard crash with no useful error message. For production code on arbitrarily-sized input, prefer iteration. For algorithms where the recursion depth is bounded (like tree traversals with $O(\log n)$ depth), recursion is fine.

Aside When recursion is genuinely better

Flat lists rarely benefit from recursion — iteration wins on every axis except elegance. Where recursion pays off: trees (chapter 6), graphs, divide-and-conquer sorts (merge sort's "split the list, sort each half"). The unifying property is branching: when each step produces multiple subproblems, the recursive structure is essential. One-way linked lists branch only once per step, which is why iteration is almost always the right tool here.

Key idea Before you move on

Write both versions once for each algorithm in this chapter. Not because you'll use the recursive ones in real code, but because the back-and-forth forces you to see the same algorithm from two angles.

The loop variant and the recursive variant are always the same algorithm — only the bookkeeping differs. Once you can translate between them freely, the rest of the course (especially trees and graph traversal) becomes much easier.

13 4.13 Stack ADT

Stacks are the first "restricted list" ADT. Where a list lets you touch any element, a stack lets you touch *one*: the top. That single restriction is the entire point. It buys you a mental model (LIFO, last-in-first-out), a suite of cheap operations (everything $O(1)$), and a surprisingly long list of real-world algorithms that just fall out of it.

Definition — Stack

A linear ADT with access restricted to one end, called the *top*. Insertion (*push*) and removal (*pop*) happen only at the top, so the most recently inserted element is always the first to come out — last-in, first-out (LIFO).

The five operations

Op	Semantics	Cost
`push(x)`	Add x at top	$O(1)$
`pop()`	Remove and return top	$O(1)$
`peek()`	Return top without removing	$O(1)$
`empty()`	True iff no elements	$O(1)$
`size()`	Number of elements	$O(1)$

All five are *guaranteed* $O(1)$ — that's not a coincidence, it's the contract. A data structure that calls itself a stack but has $O(n)$ push is mislabeled; the whole reason stacks exist is to be cheap.

Gotcha Pop/peek on empty is undefined

The ADT contract says nothing about what `pop()` or `peek()` does on an empty stack. Different implementations choose differently: `std::stack::pop` is UB (nothing returned, no throw); Python's `list.pop()` raises `IndexError`; some textbook versions return a sentinel. Always check `empty()` first, or use whatever error-signaling mechanism your library provides. Treat this like dereferencing a null pointer — the language doesn't stop you, but the consequences are yours.

C++: `std::stack`

C++ provides `std::stack` as a container *adapter* — a wrapper around an underlying container that exposes only the stack operations. The underlying container defaults to `std::deque` but can be any container that supports `push_back`, `pop_back`, and `back`.

Example `std::stack` basics

```
#include <stack>
#include <iostream>

std::stack<int> s;
```

```
s.push(7); s.push(14); s.push(9); s.push(5);

while (!s.empty()) {
    std::cout << s.top() << ' ';  // peek
    s.pop();                      // remove
}
// Output: 5 9 14 7   (LIFO)
```

Notice `top()` is the STL's name for `peek()`, and `pop()` returns `void` — you must call `top()` before `pop()` if you want the value. This split is deliberate: it avoids forcing a copy on every pop, which matters for stacks of expensive-to-copy types.

Key idea When do you pick stack over list?

Use a stack when your algorithm only ever needs the most-recent item. That sounds restrictive, but it's *exactly* the shape of: function calls (the call stack), parsing (parenthesis matching, shunting-yard), undo/redo, backtracking search, DFS graph traversal, expression evaluation, and reversing a sequence. If you catch yourself using `push_back` / `pop_back` on a `vector` and never touching any middle element, you have a stack — consider using `std::stack` to make the intent obvious and prevent accidental misuse.

Why restrict access at all?

A stack has strictly *fewer* operations than a list. That seems strictly worse. The payoff is threefold:

1. **Speed guarantee.** A list has $O(n)$ random access; a stack's operations are all $O(1)$. The restriction is what makes that guarantee possible.
2. **Code clarity.** A function that takes a `stack<T>&` advertises its intent: "I will use this LIFO." A function taking `vector<T>&` is making no such promise.
3. **Implementation freedom.** Once you commit to LIFO semantics, the implementer can swap in any underlying container (array, DLL, circular buffer) without breaking callers.

Aside The stack is the call stack is the stack

The term "stack" in "call stack" isn't a coincidence — it *is* the same ADT. When a function calls another, a stack frame is pushed. When it returns, the frame is popped. The recursive reverse-traversal from section 4.12 worked because the call stack *is* a stack, and stacks reverse things. Any iterative algorithm you can write with an explicit stack, you can also write recursively (and vice versa). This equivalence is the backbone of most tree and graph algorithms in later chapters.

14 4.14 Stacks via Linked Lists

Section 4.13 defined what a stack is. This section picks one concrete implementation: a singly-linked list where the head *is* the top. Every stack operation becomes a list operation you already know.

The mapping

Stack op	List op (head = top)	Cost
`push(x)`	`prepend(x)`	$O(1)$
`pop()`	Read head, remove head	$O(1)$
`peek()`	Read `head->data`	$O(1)$
`empty()`	`head == nullptr`	$O(1)$
`size()`	Counter updated on push/pop	$O(1)$

The key design choice is **head-as-top**. Why not tail? Because in a *singly*-linked list, prepending is $O(1)$ and appending-then-removing is $O(n)$ without a tail pointer (you'd still need to walk to find the new tail after popping). Putting the top at the head lines the cheap operations up perfectly.

Key idea Choose the end that makes both sides cheap

For any restricted ADT (stack, queue, deque), the "which end" decision is driven by which operations have to stay $O(1)$. For a singly-linked stack: head, because prepend and remove-head are both $O(1)$. For a singly-linked queue: head for dequeue, *tail* for enqueue — which is why a queue needs a tail pointer on an SLL but a stack doesn't. Section 4.15 will hit this exact wall.

Implementation

Example Singly-linked stack

```
template <typename T>
class Stack {
    struct Node { T data; Node* next; };
    Node*  head_ = nullptr;
    size_t n_    = 0;

public:
    ~Stack() { while (!empty()) pop(); }

    bool   empty() const { return head_ == nullptr; }
    size_t size()  const { return n_; }

    void push(const T& x) {
        head_ = new Node{x, head_};  // prepend
        ++n_;
    }

    T pop() {
        // caller's responsibility: !empty()
        Node*  top = head_;
        T      val = std::move(top->data);
        head_      = head_->next;
        delete top;
        --n_;
        return val;
    }
```

```
    const T& peek() const { return head_->data; }
};
```

A few things worth pointing out:

- **Destructor walks and frees.** A stack owns its nodes. Without the destructor, every `Stack` that goes out of scope while non-empty leaks the whole chain.
- **`n_` counter.** Keeping `size()` $O(1)$ requires maintaining a counter. Walking the list every time would make `size()` $O(n)$, violating the ADT contract.
- **`pop` returns by value; `peek` by const-reference.** Pop has to return because the node is about to be deleted, so you can't hand out a reference into freed memory. Peek can return a reference because the node is still alive.

Gotcha Copy/move semantics are easy to get wrong

A class that owns heap memory via a raw pointer needs the Rule of Three (destructor, copy constructor, copy assignment) or Rule of Five (add move versions). If you don't write them, C++ generates shallow copies — two stacks will share node pointers and double-free on destruction. The clean fix: hold nodes with `std::unique_ptr<Node>` so ownership is automatic, or just delegate to `std::list` / `std::forward_list` underneath.

Linked list vs. array backing

A stack doesn't *have* to be a linked list. A dynamic array (`std::vector`) with push-back-as-push and pop-back-as-pop is often faster in practice because the elements are contiguous and the CPU prefetches. The tradeoff:

Metric	Linked list	Dynamic array
push worst case	$O(1)$	$O(n)$ (on resize)
push amortized	$O(1)$	$O(1)$
pop	$O(1)$	$O(1)$
Cache friendliness	Poor	Excellent
Memory overhead / elem	1 pointer	0–1 slot (slack)
Handles huge sizes	Any	Any

For almost every use case, array-backed is the right default — which is why `std::stack` defaults to `std::deque` (array-ish under the hood) rather than `std::list`. The linked-list version shines when you need *guaranteed* worst-case $O(1)$ with no amortization.

Aside Amortization, briefly

A `vector` push is $O(1)$ on average even though occasional resizes are $O(n)$. The trick: when the vector doubles its capacity, the n work is spread across the next n cheap pushes, so the *amortized* cost is still $O(1)$. Linked lists don't need this argument — every push is $O(1)$, full stop. For real-time systems where a single $O(n)$ spike would miss a deadline, the linked list's predictable timing wins.

Key idea The bigger lesson

Notice what just happened: you got stack → done, for free, by renaming a handful of list operations. That's the point of layered ADTs. You design the list once, stress-test it, and then every restricted list (stack, queue, deque) is a ten-line wrapper that inherits all the correctness work. Building bigger systems from smaller, already-correct ones is the core abstraction skill of this course.

15 4.15 Queue ADT

A queue is the stack's twin. Where a stack is LIFO (last-in, first-out), a queue is *FIFO* (first-in, first-out). Both are restricted list ADTs with exactly two access points — the only difference is whether you remove from the same end you insert at (stack) or the opposite end (queue).

Definition — Queue

A linear ADT with insertion at one end (the *rear* / *back*) and removal at the other (the *front*). Order is preserved: the item pushed first is the item popped first — first-in, first-out (FIFO). British English calls a line at a shop a "queue"; that is exactly the mental model.

The five operations

Op	Semantics	Cost
`push(x)` / `enqueue`	Add `x` at rear	$O(1)$
`pop()` / `dequeue`	Remove and return front	$O(1)$
`peek()` / `front`	Return front without removing	$O(1)$
`empty()`	True iff no elements	$O(1)$
`size()`	Number of elements	$O(1)$

Every operation is $O(1)$, same as a stack. The restriction (only two access points) is what makes this achievable — a general list cannot guarantee $O(1)$ on both ends without extra structure.

Key idea Stack vs. queue: a one-line summary

A stack pops what it just pushed. A queue pops what it pushed the longest time ago. Everything else about the two ADTs is the same: same shape (linear), same operation count (two active ends), same cost ($O(1)$ across the board). The semantic difference — LIFO vs. FIFO — is the only thing that actually changes.

C++: `std::queue`

`std::queue` mirrors `std::stack`: a container adapter with a default underlying container (`std::deque`) and a minimal interface. The names are slightly different because a queue has two ends.

Example `std::queue` basics

```
#include <queue>
#include <iostream>

std::queue<int> q;
q.push(7); q.push(14); q.push(9); q.push(5);
```

```
while (!q.empty()) {
    std::cout << q.front() << ' ';  // peek
    q.pop();                        // remove
}
// Output: 7 14 9 5   (FIFO --- insertion order)
```

Notice the output is in *insertion* order, not reversed like a stack. Also notice `std::queue` exposes both `front()` and `back()`; you can peek at either end but only pop from the front and push to the back.

Where queues show up

Use case	What the queue represents
BFS graph traversal	Frontier of nodes to visit
Scheduling (OS, jobs)	Tasks waiting to run
Producer/consumer	Work items, producer pushes, consumer pops
Message passing	Inbox (network, IPC)
Printer / print spool	Jobs queued for the device
Buffered I/O	Bytes waiting to be read or written

The unifying theme: *"things arrive over time; serve them in arrival order."* That is a much more common pattern than LIFO, which is why queues are everywhere the moment you leave single- threaded algorithm land.

Aside BFS uses a queue; DFS uses a stack

Graph breadth-first search with a queue explores nearer nodes first. Depth-first search with a stack (or recursion, which *is* a stack) explores a path as far as possible before backtracking. The only difference between the two algorithms is which ADT holds the frontier. Same code, swap the data structure, get a different traversal order. This is one of the cleanest illustrations of why "choice of ADT" is an algorithmic choice.

Why a queue needs two pointers on a linked list

On a singly-linked list, pop-from-front is cheap (just advance `head`) but push-to-back is $O(n)$ — unless you keep a tail pointer. A stack's linked implementation needs only the head, but a queue needs both head and tail because the two operations live at opposite ends.

Gotcha Don't forget to update tail on the last pop

When a queue empties, `head` becomes `nullptr` but you must also set `tail` to `nullptr` — otherwise the next push will dereference a pointer to a freed node. This is the most common bug in handwritten queue code, and it only shows up in the empty-then-fill again path, which is easy to miss in testing. Section 4.16 walks through the full implementation.

Key idea Preview: the other queues

The basic FIFO queue is the starting point, not the end. Later sections and courses will add: *priority queue* (pop the *smallest* element, not the oldest — implemented with a heap), *deque* (push and pop from

both ends — implemented with a doubly-linked list or a ring buffer), and *circular queue* (fixed-capacity FIFO using modular arithmetic on an array). Each is a small variation on the same "ordered work list" theme.

16 4.16 Queues via Linked Lists

The queue-via-linked-list implementation is structurally almost identical to the stack version from 4.14, but with one critical addition: the queue needs a *tail* pointer so push-to-back stays $O(1)$.

The mapping

Queue op	List op (head = front, tail = rear)	Cost
`push(x)`	`append(x)` via tail pointer	$O(1)$
`pop()`	Read `head`, remove head	$O(1)$
`peek()`	Read `head->data`	$O(1)$
`empty()`	`head == nullptr`	$O(1)$
`size()`	Counter updated on push/pop	$O(1)$

The design is **head-as-front, tail-as-rear**. Pop removes from the head ($O(1)$: advance `head`). Push adds at the tail ($O(1)$ *only* if we keep a tail pointer).

Key idea The tail pointer is the whole trick

Without a tail pointer, push is $O(n)$ — you'd have to walk the whole list every time to find the end. *With* a tail pointer, it's two pointer writes: set the old tail's `next` to the new node, then set `tail` to the new node. That one extra pointer is the difference between a useful data structure and a useless one.

Implementation

Example Singly-linked queue with head and tail

```
template <typename T>
class Queue {
    struct Node { T data; Node* next; };
    Node*  head_ = nullptr;
    Node*  tail_ = nullptr;
    size_t n_    = 0;

public:
    ~Queue() { while (!empty()) pop(); }

    bool   empty() const { return head_ == nullptr; }
    size_t size()  const { return n_; }

    void push(const T& x) {
        Node* node = new Node{x, nullptr};
        if (tail_) tail_->next = node;  // link old tail
        else       head_       = node;  // was empty
        tail_ = node;
```

```
        ++n_;
    }

    T pop() {
        // caller's responsibility: !empty()
        Node* front = head_;
        T     val   = std::move(front->data);
        head_ = head_->next;
        if (!head_) tail_ = nullptr;    // queue now empty
        delete front;
        --n_;
        return val;
    }

    const T& peek() const { return head_->data; }
};
```

Two lines deserve close attention:

- **The empty-push case.** When `tail_` is `nullptr`, the queue is empty; the new node is simultaneously the head and the tail, so both pointers must be set. Missing this turns a push on an empty queue into a silent no-op.
- **The empty-pop case.** When the pop removes the last element, `tail_` must be reset to `nullptr` as well. Missing this leaves a dangling pointer: the next push will write to `tail_->next`, which is a pointer into freed memory.

Gotcha Dangling tail is the classic bug

The most common bug in handwritten queues is forgetting the `if (!head_) tail_ = nullptr;` line. The bug does not trigger on the first few operations. It triggers on "push, pop, push" — and even then only when the list has reached length zero. If your test suite never exercises that path you will ship the bug.

Why not a doubly-linked list?

A DLL would work, but it's overkill. A queue never traverses backward, never inserts in the middle, and never removes from anywhere other than the head. The `prev` pointer on every node would be dead weight — pure memory overhead with no corresponding operation to justify it.

Operation	Needs prev?
`push(x)`	No (append at tail, only need tail->next)
`pop()`	No (remove head, only need head->next)
`peek()`	No

An SLL with head+tail is the minimum sufficient structure. That's usually the right way to choose data structures: add capability up to exactly what the operations demand, no further.

Aside What about array-backed queues?

`std::queue`'s default (`std::deque`) is not a linked list at all — it's a segmented array ("deque" = double-ended queue). For most performance-sensitive work, a circular buffer (fixed-size array with head/tail indices and modular arithmetic) beats both the linked list and the deque because it has zero allocations per push and perfect cache locality. The tradeoff is a fixed capacity, which linked queues don't have. Pick based on whether your workload has a known upper bound on queue size.

Implementing the ring buffer

The notebox above gestures at the ring buffer; it deserves a concrete implementation, since it is the structure used in OS kernels for bounded producer-consumer queues, in audio/video pipelines for fixed-latency buffering, and inside `std::deque` block-by-block.

Definition — Ring buffer (circular array queue)

A fixed-capacity array $Q[0\,..\,n-1]$ with two integer indices, head and $\mathtt{tail}$, that wrap modulo n. Items live in $Q[\mathsf{head}], Q[\mathsf{head}+1], \ldots, Q[\mathtt{tail}-1]$ in circular order. Enqueue writes at $\mathtt{tail}$ and advances $\mathtt{tail} \leftarrow (\mathtt{tail}+1) \bmod n$; dequeue returns $Q[\mathsf{head}]$ and advances $\mathsf{head} \leftarrow (\mathsf{head}+1) \bmod n$. Both are $O(1)$ with zero allocation.

Example Ring-buffer queue in C++

```cpp
template <typename T, size_t N>
class RingQueue {
    T   buf_[N];
    size_t head_ = 0;
    size_t tail_ = 0;
    size_t sz_   = 0;     // explicit size: avoids head==tail ambiguity
public:
    bool empty() const { return sz_ == 0; }
    bool full()  const { return sz_ == N; }
    size_t size() const { return sz_; }

    bool enqueue(const T& x) {
        if (full()) return false;     // caller decides: drop or block
        buf_[tail_] = x;
        tail_ = (tail_ + 1) % N;
        ++sz_;
        return true;
    }

    bool dequeue(T& out) {
        if (empty()) return false;
        out  = buf_[head_];
        head_ = (head_ + 1) % N;
        --sz_;
        return true;
    }
};
```

Gotcha Full-vs-empty disambiguation

`head` == `tail` is ambiguous: it means *both* "empty" and "full" (after n enqueues with no dequeues, the write pointer has wrapped all the way around to meet the read pointer). Two standard fixes:

- **Track size explicitly** (used above): one extra `size_t sz_`, no wasted slots. Simplest.
- **Leave one slot unused**: declare full when $(\texttt{tail} + 1) \bmod n == \texttt{head}$. Saves the size counter but capacity becomes $n - 1$. CLRS Ch 10.1 uses this.

Pick one; never pick neither.

Aside Why ring buffers dominate in low-latency code

The ring buffer's appeal is structural, not just performance: fixed capacity at compile time, contiguous memory (cache-friendly), zero heap allocation per operation, and a trivially mappable single-producer-single-consumer lock-free variant (separate cache lines for head and tail, atomic loads/stores — no mutex). When queue size has a known upper bound and latency matters, this is the implementation to reach for, not the linked-list version of §4.16 and not `std::queue`.

Key idea Comparing stack 4.14 to queue 4.16

Side by side: the stack's push was `prepend`, its pop was `remove-head`. The queue's push is `append`, its pop is `remove-head`. The *only* structural difference between the two is which end the push happens at — everything else (pop location, head pointer, size counter, empty check, destructor) is identical. The two ADTs are more similar than they are different; the semantics (LIFO vs. FIFO) emerge entirely from that one choice of end.

17 4.17 Deque ADT

A *deque* (pronounced "deck," short for *d*ouble- *e*nded *que*ue) is the union of a stack and a queue. Where a stack restricts you to one end and a queue to two specific ends (front for pop, back for push), a deque lets you push and pop at *either* end. That flexibility costs nothing in asymptotic time: all four operations remain $O(1)$.

Definition — Deque

A linear ADT supporting insertion and removal at both the front and the back, each in $O(1)$. A deque generalizes both stacks (use only one end) and queues (push at one end, pop at the other) in a single structure.

The eight operations

Op	Semantics	Cost
`push_front(x)`	Insert x at front	$O(1)$
`push_back(x)`	Insert x at back	$O(1)$
`pop_front()`	Remove and return front	$O(1)$
`pop_back()`	Remove and return back	$O(1)$
`peek_front()`	Return front without removing	$O(1)$
`peek_back()`	Return back without removing	$O(1)$
`empty()`	True iff no elements	$O(1)$
`size()`	Number of elements	$O(1)$

The ADT hierarchy, visualized

ADT	Push front	Push back	Pop front	Pop back
Stack		•		•
Queue		•	•	
Deque	•	•	•	•

A stack is a deque that only uses one end. A queue is a deque that uses opposite ends. *Every* stack and queue operation is also a valid deque operation — deques strictly generalize both.

Key idea Why not just always use a deque?

Because the restrictions on stacks and queues are *features*, not bugs. A function that takes a `stack&` is declaring "I will only ever push/pop from the top"; the type system enforces that. Swap in a deque and you've invited bugs where future maintainers accidentally use `push_front` and break the LIFO invariant. Use the most restrictive ADT that satisfies your algorithm — it prevents misuse and self-documents intent.

C++: `std::deque`

`std::deque` is a full container, not an adapter. It supports random-access iterators, `operator[]`, and the standard container interface on top of the eight deque ops.

Example `std::deque` basics

```
#include <deque>
#include <iostream>

std::deque<int> d;
d.push_back(7);      // d: 7
d.push_front(14);    // d: 14 7
d.push_front(9);     // d: 9 14 7
d.push_back(5);      // d: 9 14 7 5

std::cout << d.front() << ' ' << d.back() << "\n"; // 9 5
d.pop_back();        // d: 9 14 7
d.pop_front();       // d: 14 7
std::cout << d[0] << "\n"; // 14  --- random access works
```

Aside `std::deque` is *not* a linked list

Despite "deque" suggesting a linked-list double-ended queue, `std::deque` in libstdc++/libc++ is a segmented array — a dynamic array of fixed-size chunks. Front and back push allocate a chunk at most occasionally (amortized $O(1)$), and random access is $O(1)$ because the chunk pointer array acts as a two-level index. This is why `std::stack` and `std::queue` default to `std::deque`: it gives them cheap insertion/removal at both ends with better cache behavior than a linked list.

Linked-list implementation: the DLL's payoff

A deque on a linked list is where the doubly-linked list finally earns its keep. Think about it:

Operation	SLL cost	DLL cost
`push_front`	$O(1)$	$O(1)$
`push_back`	$O(1)$ (with tail)	$O(1)$
`pop_front`	$O(1)$	$O(1)$
`pop_back`	$O(n)$ (no prev!)	$O(1)$

The fourth row is the killer. To pop the back of a singly-linked list, you have to walk the list to find the new tail (the second-to-last node), because there's no `prev` pointer to back up from the current tail. That walk is $O(n)$ — game over for the ADT contract.

Gotcha You cannot build an $O(1)$ deque on an SLL

Not even with a tail pointer. Not even with clever bookkeeping. The pop-back operation fundamentally requires reaching the second-to-last node, which on an SLL means walking from the head. If anyone suggests an "SLL-based deque," they either have an amortized scheme in mind or they have not thought it through. A deque demands a DLL (or an array).

Key idea Why this chapter built toward this section

Look back at the order of topics: list → SLL → DLL → stack → queue → deque. That progression isn't arbitrary. The stack worked on an SLL. The queue worked on an SLL (with a tail pointer). The deque *requires* a DLL to preserve its cost contract — the symmetric access pattern on both ends needs symmetric pointers on every node. This section is where all the "extra pointer per node" discussion from 4.6 finally earns its cost.

18 4.18 Array-Based Lists

The chapter has spent 17 sections on pointer-based lists. This one finally asks: *what if we implemented the list ADT on a plain old array instead?* The answer reshuffles the cost table. Array-based lists win on random access and cache performance, and lose on insertion/removal at the front. Neither approach is universally better — they have complementary strengths.

Definition — Array-based list

A list ADT implemented on top of a contiguous array. A *dynamic* array-based list allocates more space than the current length, and resizes (reallocates larger, copies) when full. A `length` counter tracks how many slots are in use; the gap between `length` and `capacity` is the available headroom.

The layout

used				unused capacity			
`[0]`	`[1]`	`[2]`	`[3]`	—	—	—	—
45	84	12	78				

`length = 4`, `capacity = 8`. Elements live at indices `[0..length)`; the rest is reserved space that doesn't need to be copied on the next append.

Append: amortized $O(1)$

Example Append to a dynamic array list

```
void append(List& L, int value) {
    if (L.length == L.capacity) {
        resize(L, L.capacity == 0 ? 1 : L.capacity * 2);
    }
    L.array[L.length++] = value;
}

void resize(List& L, size_t newCap) {
    int* p = new int[newCap];
    for (size_t i = 0; i < L.length; ++i) p[i] = L.array[i];
    delete[] L.array;
    L.array    = p;
    L.capacity = newCap;
}
```

A single append is *usually* $O(1)$: assign into `array[length]`, increment length. But when the array is full, the resize step is $O(n)$: allocate, copy every element, free the old array. So is append $O(1)$ or $O(n)$?

Key idea Amortized analysis: doubling saves you

Start with capacity 1. After appending n elements, the total copy work is $1 + 2 + 4 + \ldots + n/2 + n = 2n - 1$. That is $O(n)$ work spread across n appends, so the *average* per append is $O(1)$. This is **amortized** $O(1)$: any individual append might be slow, but over a long sequence the cost averages out. The magic number is the doubling factor — grow by a constant factor (not a constant amount) or the argument breaks.

Gotcha Growing by +1 is the classic mistake

If you resize by adding a fixed amount (say, `capacity += 10`) every time you fill up, appending n elements costs $O(n^2)$ total, not $O(n)$. Every tenth append triggers a full copy, and those copies grow linearly. Always grow by a *factor* (1.5x, 2x — both work), never an additive constant.

Prepend and insert: the expensive operations

Appending is cheap because the free space lives at the end. Prepending is $O(n)$ because every existing element has to shift right by one to make room at index 0. Inserting after index k shifts everything in $[k + 1, \text{length})$ — worst case $O(n)$, best case (insert at the end) $O(1)$.

Example Array prepend: linear shift

```
void prepend(List& L, int value) {
    if (L.length == L.capacity) resize(L, L.capacity * 2);
    for (size_t i = L.length; i > 0; --i) {
        L.array[i] = L.array[i - 1];        // O(n) shift
    }
```

```
    L.array[0] = value;
    ++L.length;
}
```

The full cost comparison

Operation	Array-based	Linked list
`append`	$O(1)^*$	$O(1)$ (with tail)
`prepend`	$O(n)$	$O(1)$
`insert at index` k	$O(n-k)$	$O(n)$ to find, $O(1)$ to insert
`remove at index` k	$O(n-k)$	$O(n)$ to find, $O(1)$ to remove
`access by index`	$O(1)$	$O(n)$
`search by value`	$O(n)$	$O(n)$
`append (at end)`	$O(1)^*$	$O(1)$
Cache friendliness	Excellent	Poor
Memory overhead per element	0–1 slot	1–2 pointers

* amortized

The two structures have *complementary* strengths. Array- based wins on index access, cache behavior, and append. Linked wins on front operations and interior insertion once the position is known. Picking between them is an algorithmic choice driven by what operations dominate.

Key idea The default is `std::vector`

In practice, C++ programmers reach for `std::vector` (dynamic array) first and only use `std::list` when they have a specific reason. The reasons: $O(n)$ front/middle operations are rarely actually the bottleneck, cache performance matters more than big-O for small-to-medium n, and memory overhead per element matters too. "Always use a linked list for frequent insertions" is a textbook myth that doesn't survive contact with modern hardware. Profile first.

Aside C++: `std::vector` is the array-based list

`std::vector<T>` is precisely this data structure, with a few extras: exception safety, move semantics, allocators, iterators, `emplace_back` for in-place construction, and `shrink_to_fit` for giving memory back. Its growth factor is implementation-defined — libstdc++ uses 2x, MSVC uses 1.5x. The 1.5x choice is marginally more memory-efficient (the old buffer could be reused after a few rounds) but the difference rarely matters. For any array-based list task in this course, use `std::vector` unless you're explicitly implementing one from scratch.

Gotcha Iterator / pointer invalidation on resize

When a `std::vector` resizes, every pointer, reference, and iterator into it becomes invalid — the old memory is freed and the elements live at fresh addresses. Code like `auto& x = v[0]; v.push_back(...); x = 5;` is undefined behavior if the push triggered a resize. Linked lists don't have this problem (nodes stay where they were), which is one real use case for preferring them.

List-variant decision matrix

The four realizations of the List ADT covered in this chapter – **dynamic array** (`std::vector`), **singly linked list** (SLL), **doubly linked list** (DLL), and **sentinel DLL** (or `std::list`) – trade cost in predictable ways. This matrix is the decision you'll actually make on assignments.

Key idea Cost of each List-ADT operation

All entries are worst-case unless marked. n = list length.

Operation	Array	SLL	DLL	Sentinel DLL
Access $v[i]$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Append (back)	$O(1)^*$ amort.	$O(1)$ w/ tail	$O(1)$ w/ tail	$O(1)$
Prepend (front)	$O(n)$ shift	$O(1)$	$O(1)$	$O(1)$
Insert after node	$O(n)$ shift	$O(1)$	$O(1)$	$O(1)$
Insert before node	$O(n)$ shift	$O(n)$	$O(1)$	$O(1)$
Remove node (have ptr)	$O(n)$ shift	$O(n)$ find prev	$O(1)$	$O(1)$
Remove by value	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Reverse traversal	$O(n)$	$O(n)$ reconstruct	$O(n)$	$O(n)$
Memory per element	$\sim$ 4–8 B	+ 1 pointer	+ 2 pointers	+ 2 pointers
Cache friendliness	excellent	poor	poor	poor

Key idea Which variant, for which job?

Read top-down; take the first match.

Do you need $O(1)$ indexing ($v[i]$)? $\rightarrow$ **array-based** (`std::vector`). Nothing else gives you random access. Accept $O(n)$ middle inserts.

Is the data mostly appended/consumed at the *ends* only? (Stack: push/pop one end. Queue: enqueue one end, dequeue the other. Log buffer: append, scan.)
$\rightarrow$ **array-based** for stacks (`std::stack` $\rightarrow$ `std::deque`). **Array-based deque** or **linked list** for queues. `std::queue` uses `std::deque`.

Are you doing many mid-list inserts/removes, AND you already hold a pointer/iterator to the node (e.g., LRU cache, intrusive list in kernel code)?
$\rightarrow$ **DLL** (or **sentinel DLL**). This is the scenario linked lists win; every other scenario, arrays usually win.

Do you need bidirectional traversal? $\rightarrow$ **DLL** or **sentinel DLL**. SLL only walks forward; walking backward means rebuilding a reverse list.

Do you want `insertBefore` as a primitive, or do the empty/first/last cases in your code feel gross?
$\rightarrow$ **sentinel DLL**. The dummy head/tail nodes make every splice look the same; no special-casing.

Is every byte precious (embedded, cold cache)? $\rightarrow$ **SLL**, not DLL: one pointer per node, not two. And consider an **array-based list** anyway – contiguous memory often wins even on "slow at middle insert" workloads because of cache.

Still unsure? $\rightarrow$ **array-based list** (`std::vector`). It's the right default. Linked lists are special-purpose; reach for them when you have a concrete reason (pointer stability, intrusive data, O(1) splicing).

Gotcha The "linked lists are always faster for middle insert" myth

The Big-O says so; real benchmarks often disagree. Inserting into a `std::vector` at position k is $O(n)$ shifts of small contiguous bytes – extremely cache-friendly. Inserting into a `std::list` is $O(1)$ but requires walking to position k first, pointer-chasing non-contiguous nodes – extremely cache-hostile. Below roughly 10^4 elements, vector insert-at-middle usually wins. Above it, it depends on access pattern. **Measure, don't assume.**

Key idea Chapter 4 wrap-up

You've now seen two complete implementations of the list ADT — one pointer-based, one array-based — and three restricted variants (stack, queue, deque) layered on top. The unifying theme: **cost is a function of layout**, not of semantics. Two data structures with the same API can have wildly different performance characteristics because their memory layout makes different operations cheap. Chapter 5 moves from linear containers to hierarchical ones (trees), where the layout decisions get even more interesting.

What this connects to

Aside Forward links to later chapters

- **Chapter 5 (hash tables).** Chaining is implemented as an array of *linked lists*: the node + pointer idiom from this chapter is reused verbatim as the overflow chain per bucket.
- **Chapter 6 (trees, BSTs).** The node `struct` generalizes from one `next` pointer to two (`left`, `right`). Tree traversal is linked-list traversal with a branching factor of 2.
- **Optional ch. 10 (graphs).** Adjacency-list graph representation is `vector<vector<int>` – i.e., a vector of lists. The cost arguments here tell you when to prefer adjacency matrix instead.
- **Stack / queue reuse everywhere.** DFS uses an explicit stack (or the call stack). BFS uses a queue. Expression parsing, undo systems, and recursion unwinding all lean on the Stack ADT from 4.13.

Companion materials. One-page cheat sheet: `cheat_sheets/ch_4.tex` (includes the decision matrix in compact form). Practice prompts (problem → pseudocode → C++): `practice_prompts/ch_4.md`.